

Docker Notes: Set 4

Layer best practices, container runtime, tagging and pushing images, image lifecycle, and Docker bridge networking

1. Image Layer Best Practices

Every Docker build reuses cached layers until it reaches the first instruction that changed; **that layer and everything after it is rebuilt**. Good Dockerfile design is about controlling that.

- **Group COPY and RUN wisely**: copy only what a step needs, so one changed file does not invalidate everything.
- **Put rarely-changing lines early** (base image, dependencies) and often-changing lines (your source code) last.
- **Avoid unnecessary RUN commands** to keep the layer count and image size down.

Goal	What is better?	Why?
Fast builds	More layers (smart caching)	Docker skips unchanged layers
Smaller images	Fewer layers, clean builds	Fewer intermediate files and layers
Maintainability	Readable layers	Easier debugging and edits
Layer reusability	Split steps logically	Base layers can be reused/cached

Note: Fast builds and small images pull in opposite directions. The balance: **group commands that always change together** into one RUN, and **split steps that change at different speeds** into separate layers.

Example: ordering for cache (Python app)

Poor order	Better order
<pre>FROM python:3.10-slim WORKDIR /app COPY . . RUN pip install -r requirements.txt CMD ["python", "main.py"]</pre>	<pre>FROM python:3.10-slim WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt COPY . . CMD ["python", "main.py"]</pre>

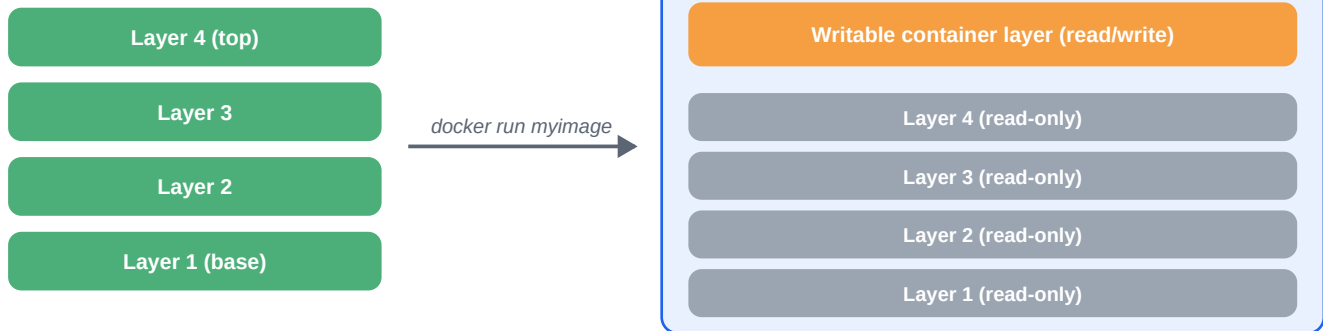
Why: in the poor order, editing any source file changes `COPY . .`, so `pip install` runs again every build. In the better order, dependencies are reinstalled only when `requirements.txt` changes, and a code edit only rebuilds the last `COPY`.

```
# Combine related RUN steps into one layer and clean up in the same step
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

2. Container Runtime

Runtime is the container execution phase. Docker creates a container from the image and runs its **CMD** or **ENTRYPOINT**. The image layers stay read-only; Docker adds a thin **writable container layer** on top, where all changes made by the running container are stored.

Docker image (read-only)



```
docker run -it --name test ubuntu bash
root@test:/# apt-get update && apt-get install -y vim
# vim is written to the writable layer only
root@test:/# exit
docker rm test
# container removed -> its writable layer (and vim) is gone
```

Note: The image itself is never changed by what a container does. Many containers can share the same image layers, each with its own writable layer. To keep changes, build a new image (Dockerfile or `docker commit`) or use volumes for data.

CMD vs ENTRYPOINT

	CMD	ENTRYPOINT
Role	Default command or default arguments	The fixed executable of the container
Override at run	Replace by adding a command: <code>docker run img other</code>	Needs <code>--entrypoint</code> ; extra args are appended to it

```
ENTRYPOINT ["echo"]
CMD ["hello"]
```

```
docker run myimg          # echo hello
docker run myimg bye      # echo bye (CMD replaced, ENTRYPOINT kept)
```

Image tag

A **tag** labels a specific version of an image after the colon. Without a tag Docker uses `latest`.

```
docker run postgres:10.10 # tag = 10.10 (exact version)
docker run postgres       # same as postgres:latest
```

3. Docker Registry: Naming, Tagging and Pushing

A **registry** stores images. Docker Hub uses a naming convention that helps with organisation, versioning and discovery:

```
[registry/][username/]repository[:tag]
```

Part	Meaning	Notes
registry	Domain of the registry	Optional; Docker Hub is the default
username	Account or organisation name	Needed for your own images on Docker Hub
repository	Image or project name	Required
tag	Version	Optional; defaults to latest

Full name	Reads as
nginx	Official image, Docker Hub, tag latest
myusername/myapp:1.0	Docker Hub account myusername, repo myapp, version 1.0
registry.example.com/devops/myapp:1.0.0	Private registry, account/project devops, repo myapp, version 1.0.0

docker tag - give an image a new name

Adds an extra name to an existing image (no copy is made; both names point to the same image). It is also how you prepare an image for pushing, because the name must include the registry and account.

Syntax:

```
docker tag SOURCE_IMAGE[:tag] TARGET_IMAGE[:tag]
```

Examples:

```
docker tag python:3.10 my-python-app          # my-python-app:latest now refers to python:3.10
docker tag myimage myusername/myapp:latest    # ready for Docker Hub
docker tag myapp registry.example.com/devops/myapp:1.0.0 # ready for a private registry
```

Note: An image must be tagged with the correct registry and account name **before** it can be pushed.

docker login - authenticate

Logs in to a registry. With no host it logs in to Docker Hub.

Syntax:

```
docker login [REGISTRY_HOST]
```

Examples:

```
docker login          # Docker Hub
docker login registry.example.com # private registry
```

docker push - upload an image

Uploads a locally tagged image to the registry. It takes **one** argument: the full tagged name.

Syntax:

```
docker push [REGISTRY_HOST/]USERNAME/REPO[:TAG]
```

Examples:

```
docker push myusername/myapp:latest
docker push registry.example.com/devops/myapp:1.0.0
```

Careful: Your note `docker push myapp myusername/myapp:latest` has two arguments, which is wrong. Tag first, then push only the tagged name.

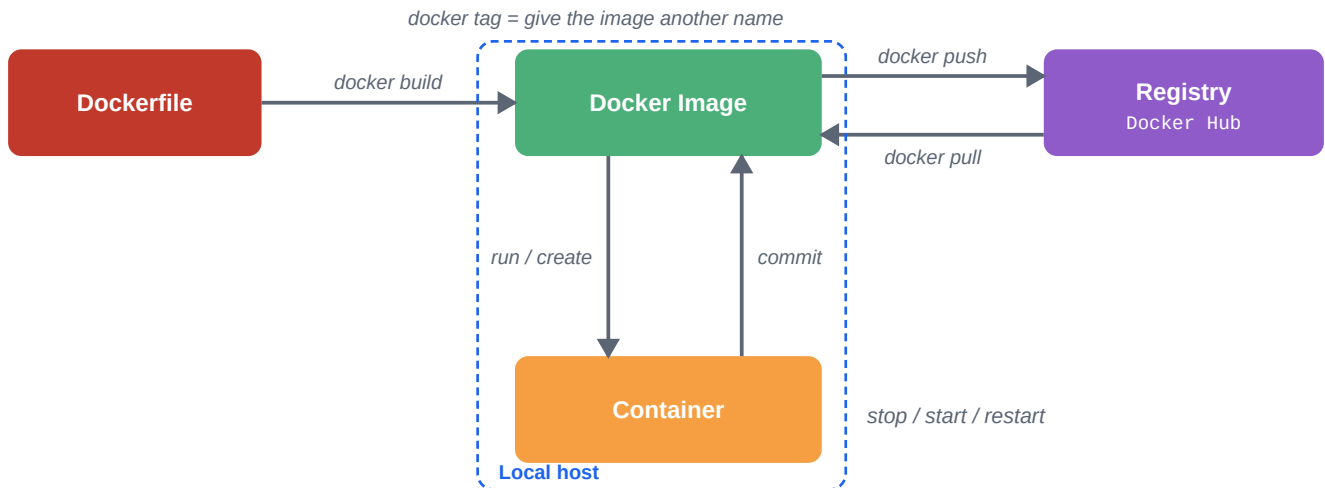
End-to-end: build, tag, push, pull

```
docker build -t myapp .          # 1. build locally
docker tag myapp myusername/myapp:1.0 # 2. add the registry-ready name
docker login                     # 3. sign in to Docker Hub
docker push myusername/myapp:1.0   # 4. upload

# on any other machine:
docker pull myusername/myapp:1.0
docker run -d myusername/myapp:1.0
```

4. Docker Image Lifecycle

This diagram ties all commands together: how a Dockerfile becomes an image, a container, and a shared image in a registry.



Command	Direction	Purpose
<code>docker build</code>	Dockerfile -> Image	Create an image from instructions
<code>docker run / create</code>	Image -> Container	Run (or just create) a container
<code>docker stop / start / restart</code>	Container	Control the container state
<code>docker commit</code>	Container -> Image	Save a container's current state as a new image
<code>docker tag</code>	Image	Add another name or version to an image
<code>docker push / pull</code>	Image <-> Registry	Upload or download images

```
docker commit test myusername/ubuntu-vim:1.0    # container 'test' -> new image
```

Tip: `docker commit` is handy for experiments, but a Dockerfile is better for real work because it is repeatable and readable.

5. Docker Bridge Networking

A **bridge network** uses a software bridge so containers on the same Docker host can talk to each other. When Docker starts it automatically creates a default bridge called **docker0**.

Network modes (drivers)

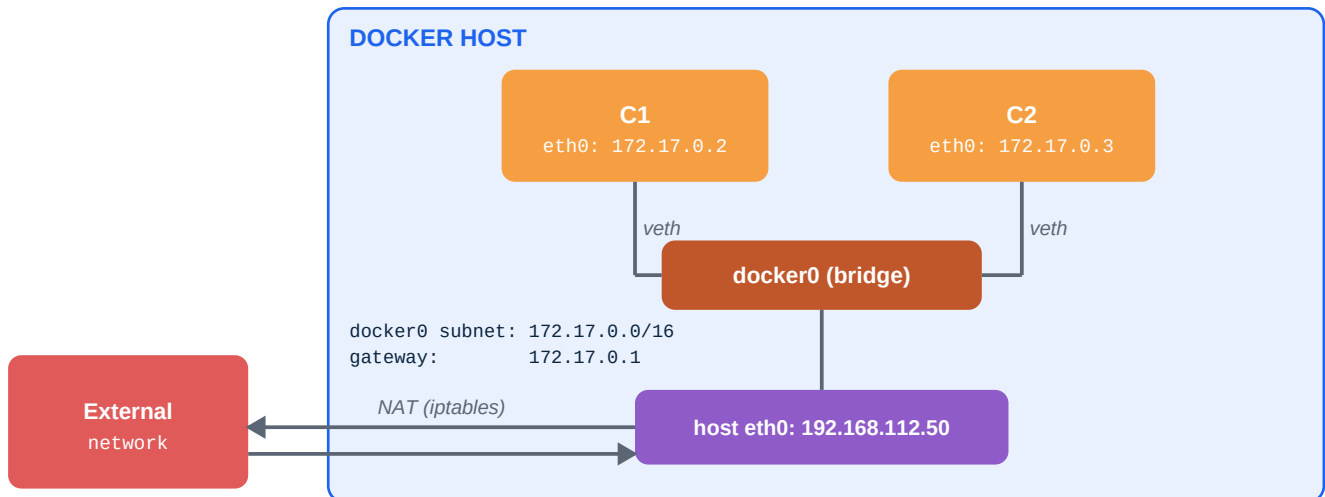
Mode	What it does
bridge (default)	Private internal network on the host. Each container gets its own IP; outbound traffic uses NAT. Publish ports with <code>-p</code> .
host	Container shares the host's network stack. No isolation, no port mapping needed.
none	No networking at all (only loopback). Fully isolated.
overlay	Connects containers across multiple Docker hosts (Swarm clusters).
ipvlan	Containers get IPs on your physical network, with fine control over addressing.
macvlan	Each container gets its own MAC address and appears as a separate device on the LAN.
user-defined	Your own network made with <code>docker network create</code> (usually a custom bridge).

Bridge mode: key facts

- Containers on the **same** bridge can communicate with each other.
- A bridge **isolates** its containers from containers on **other** bridges.
- Bridges work within a **single Docker host**.

- A container can be connected to **multiple** bridge networks at the same time.

The default bridge (docker0)

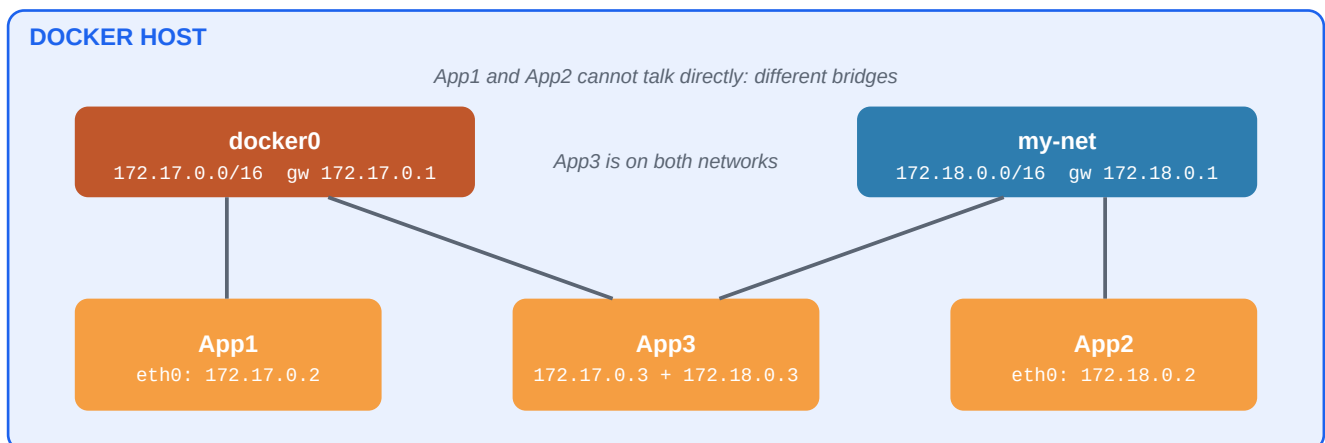


Fact	Value / meaning
Subnet	172.17.0.0/16
Gateway	172.17.0.1 (between docker0 and the host)
Default attachment	New containers join docker0 unless you choose another network
Inside each container	An eth0 interface with its own IP (for example 172.17.0.2)
On the host	A virtual interface (veth) is created for every running container and plugged into the bridge

Outbound traffic and NAT: when a container (172.17.0.2) contacts an outside server, the Docker host performs **Network Address Translation**: the source address becomes the host's address (192.168.112.50) and the reply is translated back. These NAT entries live in the host's **iptables**.

Multiple bridge networks

Besides docker0 you can create custom bridges, each with its own subnet (for example 172.18.0.0/16) and gateway. Containers on different bridges are isolated until you connect them.



Note: Default bridge vs user-defined bridge: on a user-defined bridge, containers can reach each other **by name** (built-in DNS). On the default docker0 bridge they can only use IP addresses. So for multi-container apps, create your own network.

6. Docker Network Commands

Command	Purpose
<code>ip a</code>	Show network interfaces and IPs on a Linux host (you will see <code>docker0</code> and the veth interfaces)
<code>docker network ls</code>	List all networks
<code>docker network create NAME</code>	Create a network with a given name (bridge by default)
<code>docker network create --subnet 10.0.0.0/16 NAME</code>	Create a network with a custom subnet
<code>docker network inspect NAME_or_ID</code>	Show detailed JSON: subnet, gateway, connected containers
<code>docker run --network NAME IMAGE</code>	Start a new container directly on a network
<code>docker network connect NETWORK CONTAINER</code>	Attach a running container to a network
<code>docker network disconnect NETWORK CONTAINER</code>	Detach a running container (not possible for none and host modes)

Hands-on: two containers talking by name

```
docker network ls                # bridge, host, none exist by default
docker network create my-net     # 1. create a custom bridge
docker network inspect my-net    # 2. note the subnet, e.g. 172.18.0.0/16

docker run -d --name web --network my-net nginx          # 3. start a server on my-net
docker run -it --rm --network my-net alpine ping -c 3 web
# -> replies from web: the name 'web' resolves automatically on my-net

docker run -d --name other nginx      # 4. this one joins the default docker0 bridge
docker network connect my-net other    # 5. now 'other' is on my-net too
docker network disconnect my-net other # 6. and now removed again
docker network inspect bridge         # see which containers are on docker0
docker network rm my-net              # 7. delete the network (after containers leave it)
```

Quick Reminders

- Tag before push: the image name must include your account (or registry host).
- Changes inside a container live in its writable layer; remove the container and they are gone.
- Default bridge: IP only. User-defined bridge: names work too.
- `docker network disconnect` does not work for the none and host modes.
- Port publishing (`-p 8080:80`) is what exposes a bridge-mode container to the outside; NAT only handles outbound traffic.